

# Reliable and Trustworthy Artificial Intelligence #3

## 1. Model, Dataset, and Verification Query

**Dataset.** FashionMNIST (10 classes, 28×28 grayscale, 60K/10K). 픽셀은 [0, 1] 로 스케일링하며 별도 정규화는 하지 않아  $L_\infty$  perturbation 이 "픽셀 강도  $\pm \epsilon$ " 와 직접 대응되도록 했다.

**Model.** 3 층 fully-connected ReLU 네트워크를 직접 학습시켰다.

```
input(784) -> Gemm(32) -> ReLU -> Gemm(16) -> ReLU -> Gemm(10) -> logits
```

Adam (lr=1e-3, batch=128) 으로 3 epoch 학습하여 test accuracy 83.2% 를 달성했고, ONNX (opset 12) 로 export 했다. 그래프에는 Gemm 와 ReLU 노드만 등장하며 모두 Marabou ONNX parser 가 안정적으로 지원한다. Softmax 는 의도적으로 제외하여 transcendental 함수에 대한 reasoning 부담을 없앴다.

**Verification Query.** 테스트 이미지 (index 0, label = 9 "Ankle boot") 와  $L_\infty$  반지름  $\epsilon$  에 대해,  $\epsilon$ -ball 안의 모든 입력이 같은 클래스로 분류되는지를 묻는다. 9 개의 target class  $t \in \{0, \dots, 8\}$  각각에 대해  $\epsilon$ -ball 안에서  $f_t(x') \geq f_9(x')$  를 만족하는  $x'$  가 있는지 묻는 9 개의 독립 query 로 분해된다. 모두 UNSAT 이면 robust, 하나라도 SAT 이면 그것이 곧 adversarial example 이다. 인코딩은 `resources/runMarabou.py::encode_mnist_linf` 를 차용했다.

## 2. Results and Interpretation

| eps   | verdict    | total time | comment                    |
|-------|------------|------------|----------------------------|
| 0.001 | robust     | 0.33 s     | all 9 targets UNSAT        |
| 0.005 | robust     | 0.42 s     |                            |
| 0.010 | robust     | 0.40 s     | last fully-robust radius   |
| 0.020 | not robust | 0.36 s     | SAT on target = 5 (Sandal) |
| 0.050 | not robust | 0.44 s     | SAT on target = 5 again    |

**Sharp robustness boundary.** Robustness 가  $\epsilon = 0.01$  과  $\epsilon = 0.02$  사이에서 무너진다.  $\epsilon = 0.02$  에서 Marabou 는 target 0-4 를 차례로 UNSAT 으로 증명한 뒤 target 5 (Sandal) 에서 SAT 을 반환했다. 즉 결정 기하 상 "Ankle boot" 와 가장 가까운 wrong class 가 Sandal 이라는 뜻이며, 두 클래스가 모두 신발류로 실루엣을 따라 강한 픽셀이 분포한다는 점에서 어느정도 이해되는 결과이다.

**Runtime profile.** UNSAT target 의 per-target 시간은  $0.04 - 0.07$  s,  $\epsilon = 0.02$  의 SAT case 도  $0.11$  s 로 매우 짧다. 전체 wall-clock time 이 모든  $\epsilon$  에서  $0.33 - 0.44$  s 범위로 거의 평탄하다는 점은 비용의 상당 부분이 SMT 탐색이 아니라 ONNX 파싱과 query 별 setup 에 들어가 있음을 시사한다.

### 3. Strengths and Limitations of Marabou

#### Strengths

- **Soundness** PGD/FGSM 같은 random attack 과 달리 UNSAT 응답은  $\epsilon$ -ball 안에 adversarial input 이 "존재하지 않는다" 는 진짜 증명이다. Confidence score 가 아니라 보장이 필요한 상황에서 의미가 크다.
- **Python API.** `Marabou.read_onnx`, `setLowerBound`, `setUpperBound`, `addInequality`, `solve` 만으로  $L_\infty$  workflow 가 30 줄 이내로 끝난다는 점에서 간단하다.
- **Overhead** 본 실험의  $\approx 30K$  파라미터 모델은 9 개 target 을 모두 순회해도  $\epsilon$  당 1 초 미만에 검증된다.

#### Limitations

- **Output 명세가 어색하다.** "argmax 가 그대로 유지된다" 는 first-class predicate 이 없어 robustness 를  $N - 1$  개의 inequality query 로 표현해야 한다. 10-class 에서는 괜찮지만 클래스 수가 많아지면 비효율적이다.
- **Scalability 의 천장이 낮다.** 이 정도의 작은 MLP 라도 hidden width 를 키우거나(예: `mnist2x256.nnet` 수준) convolution 을 추가하면 query 당 시간이 수십 초로 늘어나며, `resources/onnx/cifar10/` 의 KW 모델이 사실상 한계 근처에 위치해 있다.
- **라이브러리 호환성 마찰.** Pre-compiled MarabouCore 가 `GLIBCXX_3.4.32` 를 요구해 `LD_PRELOAD` 로 우회해야 했다.